

Звіт

Python-інтерпретатор

Стегня Віктора ПМі-44

Мета та основні завдання

1. Реалізувати міні-інтерпретатор підмножини Python для лабораторної роботи.
2. Побудувати лексичний аналізатор з підтримкою чисел, ідентифікаторів, ключових слів та відступів INDENT/DEDENT.
3. Реалізувати синтаксичний аналізатор методом рекурсивного спуску за заданою граматикою.
4. Підтримати арифметичні та логічні вирази, присвоєння, print(), if-else та цикл while.
5. Додати вбудовані функції sin(), cos(), int(), float(), input() та обробку помилок виконання.
6. Підготувати граматiku, тестові програми та результати роботи інтерпретатора.

Короткий опис реалізованої системи

У результаті роботи було створено консольний міні-інтерпретатор мовою Python, який обробляє спрощену мову, наближену до Python. Програма працює за класичною схемою:

- лексичний аналіз тексту програми з формуванням токенів і службових INDENT/DEDENT;
- побудова абстрактного синтаксичного дерева методом рекурсивного спуску;
- окремий етап виконання побудованого дерева в середовищі змінних.

Інтерпретатор підтримує:

- типи int, float, bool;

- арифметичні операції +, -, *, /, //, %, ** та унарні +, -;
- логічні операції and, or, not і порівняння ==, !=, <, <=, >, >=;
- виклики sin(), cos(), int(), float(), input();
- оператори присвоєння, print(), умовні конструкції if-else та цикл while.

Основні компоненти проєкту

1. Лексичний аналізатор

У файлі `mini_python_interpreter.py` клас `Lexer` реалізує розбиття вхідного тексту на токени. Підтримано числа, ідентифікатори, ключові слова, багатосимвольні операції, коментарі та контроль відступів. Під час аналізу формуються службові токени `NEWLINE`, `INDENT`, `DEDENT`, а також виявляються помилки табуляції або неправильного рівня вкладеності.

2. Синтаксичний аналізатор

Клас `Parser` побудований за методом рекурсивного спуску. Він розбирає присвоєння, `print()`, `if-else`, `while`, виклики функцій і вирази з правильним пріоритетом операцій. Окремо у файлі `grammar_part4.txt` збережено фінальну граматику реалізованої мови.

3. Модуль виконання програми

Клас `Executor` виконує побудоване синтаксичне дерево у словнику змінних. Реалізовано обчислення арифметичних та логічних виразів, роботу з функціями `sin()`, `cos()`, `int()`, `float()`, `input()`, форматування виводу через `print()` та перевірку типових помилок, зокрема ділення на нуль, використання невизначених змінних і неправильну кількість аргументів функцій.

4. Тестові приклади та результати

У каталозі `tests/` підготовлено три приклади програм, а в каталозі `results/` збережено результати їх виконання. Перевірено:

- `test1_arithmetic_if.mpy`: арифметичні вирази, `sin/cos` і умовний оператор; результат містить значення 14, 1.0, -4 та рядок 14 -4;

- test2_input_while.py: зчитування числа через input() і підрахунок суми в циклі while; для входу 5 отримується 15;
- test3_logic_nested.py: вкладений if у циклі while, перевірка логіки та роботи з float; результат: 3 2.8, False, 1 3.8.

Висновки

У ході виконання проєкту було розроблено навчальний міні-інтерпретатор підмножини Python із повним циклом обробки програми: від лексичного аналізу та синтаксичного розбору до безпосереднього виконання команд. Отримане рішення демонструє базові принципи побудови інтерпретаторів і підтверджує коректність роботи на підготовлених тестових прикладах.